

1's Complement notation

It is a notation in which the msb is a sign bit and the positive numbers are represented as msb which is sign bit as 0 and the value after that

Example : +5 is represented as 4'b0101

for negative numbers we find the ones complement of the positive value

Example : -5 is represented as 4'b0101

with ones complement which is inverting every bit causes 4'b1010.

But there are many problems

like for 0 there will be 2 representation like 4'b0000 and 4'b1000 which is +0 and -0 which is not possible in mathematics.

And in alu operations too the overflow bit should be added to the value to get the actual value. Like in 5 bit addition -3+7 we get +3= 00011 and -3 is 11100 and +7 is 00111 adding we get

```
11100
+ 00111
100011
```

actual answer is 11+1 which is 100. so there is a additional method coming in those operations thus the resource using will be high.

2's Complement notation

it is the notation in which msb is sign bit and positive numbers are represented by sign bit as 0 and other bits normally.

And the negative numbers are written by writing the positive number and then finding ones complement and then adding 1 to it.

Example -5 is written as. +5 is 0101 then ones complement is 1010 adding 1 we get 1011. that is the representation of -5. we will have a overflow if the msb bit and the overflow bit is of different bits.

Example when adding -5 and -4. their binary representation in 2's complement is 1011 and 1100 and adding we get 10111 since the overflow and msb is 1 and 0 respectively there is overflow and we can make the overflow register notice it .

The number represented will have value as 2's power of that index.

1011 value can be found out by $-2^3+2^2+2^1+2^0$
which is $-8 + 2+1$ which is -5.

also we can use the ones complement strategy invert it and add 1 to get the value like 1011 to 0100 and adding 1 we get 5 so it is -5.

Fixed Point Q's Point representation

Q's point representations is used because fractional numbers need somehow to be represented in binary bits. But the floating point is not preferred because it takes large hardware space, power and takes large clock cycles to run compared to fixed point representation.

The symbol is $Q_m.n$ where m is the bits in real part and the n is the bits in fractional part. $Q_{4.4}$ means 4 parts are real and 4 parts are fractional. In default way the msb is a sign bit. The lsb is the resolution of the qpoint.

For $q_{4.4}$ the binary equivalent value is like $-2^3 2^2 2^1 2^0$ as real part where -2^3 is the msb sign bit and the fractional part is $2^{-1} 2^{-2} 2^{-3} 2^{-4}$ and the resolution is the 2^{-4} because that is the step size between the numbers we can represent the range is from 1000_0000 to 0111_1111.

for converting a number into q format we will multiply it with the 2^n example if we need to convert 3.625 into $q_{4.4}$ we will multiply it with 2^4 and we will get answer as 58 whose binary representation is 0011 1010 so we store it as 0011.1010.

if we have 58 then we can convert it back by dividing it by 2^4 .

Shifting to left is multiplying operation. If we shift by 1 bit it is like multiplying by 2^1 . similarly shifting to right is division.

Addition in Qs point representation

for addition the q's point numbers should share same fractional part. If $Q_{4.4} + q_{6.2}$ then we must make the small fractional part same right. So after converting it like $q_{4.4}$ multiply by 16 and the $q_{6.2}$ by 4 we should shift the resultant $q_{6.2}$ number after by 4-2 to make it share same fractional bit. And after that we can perform the addition operation and by dividing the answer by 16 will give us the final answer
the max bit we should allocate is 6+1 for the real part and the 4 for the fractional part.
So the answer will $q_{7.4}$

Multiplication in Qs point representation

$q_{4.4} * q_{6.2}$ gives output result as $q_{10.6}$

example if we multiply 5.25 and 10.75 first we need to multiply 5.25 with 2^4 which gives 84 and for the 10.75 we multiply with 2^2 we get 43 now multiply both gives 3612 it is in $q_{10.6}$ representation. We should divide it by 16 for getting the answer which is 56.4375.
in case of binary the binary representation of 3612 is 0000_1110_0001_1100 and we shift it by 6 place we get 0000_1100_00.01_1100. which is binary for 56.4375.

Quantization modes

Truncation

in which we drop the lsb

it causes the current value to decrease always so it is -ve bias

for example if we have 5.54 and we need only 1 fractional part then we will make it 5.5 that is less than 5.54 same case apply to 5.59 we will make it 5.5 that is less than 5.59 so it is creating a -ve bias.

Round to 0:

for positive bits just drop lsb so it will have -ve bias so it will have round to zero

for -negative bits we will find the 2's complement to make it +ve and then truncate to make it -ve bias and then find 2's complement again and then it would become +ve bias in case of -ve devices.

Round to Nearest

we add half lsb and then truncate it.

For example in case of 5.3254 to 3 fractional bit we will add .0005 and then truncate it so 5.3259 will become 5.325 but for the 5.3256 when addign and truncating will become 5.326 which is correct in this case.

Convergent rounding

lsb is the last bit you keep

round bit is the firs bit you drop

for

010.010101 if we need only 2 fractional bits then the 2^{-2} th bit is the lsb and the first bit we are dropping is the 2^{-3} rd bit that is the rounding bit and after that from 2^{-3} is the trailing bits

if RB is 0

if round bit is 0 then it is less than the half so we can truncate it. Example here in fractional bits the half after the lsb is .011000 this is half of the lsb bit 2^{-2} . here in this case it is 0 so less than .001000 so we can truncate it

if RB is 1

if it is half like things like .011000 then if trailing bits are 0 in the case .011000 then we look at the lsb here it is .010101. here it is 1 so we should add 1 . if it is 0 then do nothing just truncate it.

And another case if it is more than half that is from .011001 onwards then add 1 to lsb. So if it is .011001 then add 1 to lsb and truncate so it becomes .10 .

Overflow Handling

wrap around

in which it doesn't consider the extra bit it would just throw away the extra bit

Ex: 7-2

```
  11
 0111
+1110
-----
10101
```

if answer is 4 bit we can just neglect the overflow so it becomes +5

here it is correct

if considered 4 bit arithmetic 2's complement

for 7+3

```
   111
   0111
+  0011
-----
  1010
```

here the answer is 1010 so its 2's complement is 0101+1 is 0110 which is -6 which is wrong

saturation mode

in which it clamps to maximum value

in this if we are using 4 bit 2's complement then the range is from -2^3-1 to $+2^3$. so -8 to +7. so if value is greater -9 and all then it would clamp to -8 similarly in positive number greater than +7 it clamps to +7.

symmetric saturation

it consider the max and min value symmetrically and then clamps

similar to saturation it clamps but it clamps symmetrically -7 to +7 is the last case.

saturation to zero

when it goes out of range it then clamps to 0 .

Floating point representation

most of the cpus running now mainly use floating point representatio rather than fixed point.

Because cpu is for general purpose and creating a floating point hardware inside processor handles those needs.

But in fpgas if we are creating filters and all then it is not necessary we need the full floating point we only need to allocate according to out needs. Sometimes we need only 8 bits with 7 bit resolution so we use q1.7 representation like that. So in order not to waste hardware we use fixed point in fpgas

fixed point have precision trade offs. In floating point too there is tradeoff but it provide much better precison and range in the standard.

In cpus if the value range is known then we can use fixed point to get better performace because in floating point every thing need to be shifted and all everytime but in fixed point no need that everytime.

FLOATING POINT

IEEE standard

1 sign bit

8 exponent bit

23 fraction bit

IEEE 754 standard

$-1^s * F * 2^E$

we store as

$E+127$ because if we have -3 and all we can just add 127 to it and store it as E . stored exponent as 124

127 chosen because 2^8 is 255 so half of it

every number needs to be shifted to be needed to represent in this representation which is a negative reason for using in fpgas.